

UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Revaldo Fiqqih Septiansyah
NPM	2428240110
Kelas	SI5C
Topik Tugas 1	8 - Hotel: Kamar
Entitas utama	Kamar
Tautan repositori	https://github.com/revaldofiqqihseptiansyah2428240110/si5c_kuis1_revaldo
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026



Identitas Topik dan Rancangan Endpoint

Data topik

Topik Tugas 1	8 - Hotel: Kamar
Entitas utama (tunggal)	Kamar
Alamat dasar	/rooms
Field data	id, nomorKamar, tipe, hargaPerMalam, kapasitas, tersedia
Field wajib saat POST	nomorKamar, tipe, hargaPerMalam, kapasitas
Query string untuk filter	tipe (contoh: /rooms?tipe=deluxe)
Data awal	3 kamar: 101 (standar), 305 (deluxe), 401 (suite)
Modul lain (jika ada)	Tidak ada

Daftar endpoint

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/rooms	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/rooms/:id	Tidak	200	Ambil satu data
3	POST	/rooms	Ya	201	Tambah data
4	PUT	/rooms/:id	Ya	200	Ubah data
5	DELETE	/rooms/:id	Ya	204	Hapus data



Bab 1 Instalasi

1.1 Instalasi project

Screenshot terminal yang memuat perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.

Gambar. Versi Node.js dan instalasi paket

Penjelasan: express adalah framework web untuk membuat server dan route; cors mengizinkan API diakses dari origin lain (misalnya client berbeda port); dotenv membaca file .env sehingga konfigurasi seperti PORT dan API_KEY tidak ditulis langsung di kode. Versi Node.js harus LTS agar kompatibel dengan versi Express 5 yang dipakai.

Bab 2 Struktur Folder dan Ringkasan Kode

2.1 Struktur folder

Gambar. Struktur folder project

Penjelasan: models/ menyimpan data dan fungsi pengolahannya; controllers/ menangani request, validasi, dan response; routes/ memetakan alamat endpoint ke fungsi controller; middlewares/ berisi logger, cekApiKey, dan errorHandler yang menyaring dan menangani request sebelum/sesudah route.

2.2 Ringkasan tugas setiap berkas

Berkas	Penjelasan
models/roomModel.js	Menyimpan array rooms (3 data awal) dan fungsi getAll, getByld, create, update, remove. Tidak memakai req dan res sama sekali, sehingga data dan logika bisnis terpisah dari HTTP.



controllers/roomController.js	Menangani request dan response: mengambil req.params/req.query/req.body, menjalankan validasi field wajib lewat fungsi validasi(), lalu memanggil fungsi model dan mengirim res.status(...).json().
routes/roomRoutes.js	Membuat express.Router() dan memetakan alamat /rooms ke fungsi controller. Route POST, PUT, dan DELETE diberi middleware cekApiKey, sedangkan GET dibuka tanpa API key.
middlewares/logger.js	Mencatat method, URL, dan waktu setiap request (console.log) lalu next(), agar aktivitas server terpantau di terminal.
middlewares/cekApiKey.js	Membandingkan header x-api-key dengan API_KEY pada .env. Jika tidak dikirim atau salah, request ditolak dengan 401 sebelum controller dijalankan.
middlewares/errorHandler.js	Menangani error terpusat: fungsi notFound mengirim 404 untuk route yang tidak ada; errorHandler mengubah JSON rusak dari express.json() menjadi respons 400 'Format JSON tidak valid', dan error lain menjadi 500.
app.js	Entry point: memuat dotenv, mendaftarkan cors, logger, express.json(), memasang router /rooms, lalu middleware notFound dan errorHandler di paling akhir.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Gambar. Server berhasil dijalankan

Penjelasan: Nomor port berasal dari file .env (PORT=3000) yang dibaca lewat process.env.PORT dengan nilai cadangan 3000. Jika port sudah dipakai aplikasi lain, server



gagal start dengan pesan EADDRINUSE; solusinya menghentikan aplikasi tersebut atau mengganti nilai PORT di .env.



Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi menjadi lima skenario berhasil (T01-T05) dan empat skenario gagal (N01-N04). Skenario berhasil dilakukan berurutan karena T04 dan T05 memakai data yang dibuat pada T03. Nilai x-api-key yang dipakai: revaldo (sesuai API_KEY pada .env).

T01 Menampilkan semua data (GET /rooms)

Permintaan	GET /rooms
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data (3 kamar)
Status hasil uji	200 OK; larik JSON 3 kamar tampil
Lulus?	Ya

Gambar. Hasil GET semua data

Penjelasan: Kode status 200 OK menandakan permintaan berhasil dan server mengembalikan larik JSON berisi seluruh kamar. Alur pemanggilannya: route router.get("/", getAllRooms) di routes/roomRoutes.js menjalankan fungsi getAllRooms di controllers/roomController.js, yang memanggil roomModel.getAll(req.query.tipe) di models/roomModel.js. Karena tidak ada query tipe, model mengembalikan seluruh array rooms.

T02 Menampilkan satu data berdasarkan ID (GET /rooms/2)

Permintaan	GET /rooms/2
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id 2
Status hasil uji	200 OK; objek kamar id 2 (305, deluxe) tampil
Lulus?	Ya

Gambar. Hasil GET berdasarkan ID

Penjelasan: ID diambil dari alamat melalui req.params.id. Nilai params selalu berupa string, sehingga perlu diubah menjadi angka dengan parseInt() agar bisa dibandingkan dengan properti id (number) pada objek di array menggunakan find(). Route GET /rooms/:id memanggil getRoomById di controller, yang memanggil roomModel.getById(id).



T03 Menambah data baru (POST /rooms)

Permintaan	POST /rooms
Header	x-api-key: revaldo Content-Type: application/json
Body	{ "nomorKamar": "Kamar Revaldo 2428240110", "tipe": "deluxe", "hargaPerMalam": 900000, "kapasitas": 3, "tersedia": true }
Hasil yang diharapkan	201 Created; objek baru beserta id 4 yang dibuat server
Status hasil uji	201 Created; 'Data berhasil ditambahkan', data id 4
Lulus?	Ya

Gambar. Hasil POST data baru

Penjelasan: Header x-api-key diperiksa oleh middleware cekApiKey sebelum controller dijalankan; tanpa key yang benar request ditolak 401. `express.json()` mengubah body teks menjadi objek JavaScript pada `req.body`. Status yang dipakai 201 (bukan 200) karena operasi ini membuat resource baru di server, sesuai semantik HTTP. Field `nomorKamar` sengaja diisi identitas saya ('Kamar Revaldo 2428240110') sebagai tanda gambar berasal dari pekerjaan saya.

T04 Mengubah data (PUT /rooms/4)

Permintaan	PUT /rooms/4
Header	x-api-key: revaldo Content-Type: application/json
Body	{ "nomorKamar": "306", "tipe": "suite", "hargaPerMalam": 1200000, "kapasitas": 2, "tersedia": false }
Hasil yang diharapkan	200 OK; objek id 4 dengan field yang sudah berubah
Status hasil uji	200 OK; tipe berubah deluxe menjadi suite, harga dan kapasitas ikut berubah
Lulus?	Ya

Gambar. Hasil PUT mengubah data



Penjelasan: Yang berubah adalah seluruh field sesuai body (nomorKamar menjadi 306, tipe menjadi suite, hargaPerMalam 1200000, kapasitas 2, tersedia false), sedangkan id tetap 4 karena id tidak diambil dari body. Perubahan dilakukan di layer model melalui `roomModel.update(id, data)` yang menimpa properti objek dengan `Object.assign`; controller hanya menangani validasi dan response.

T05 Menghapus data (DELETE /rooms/4)

Permintaan	DELETE /rooms/4
Header	x-api-key: revaldo
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons
Status hasil uji	204 No Content; body respons kosong
Lulus?	Ya

Gambar. Hasil DELETE data

Penjelasan: Status 204 No Content berarti permintaan sukses tetapi server sengaja tidak mengembalikan body apa pun, karena setelah data dihapus tidak ada informasi yang perlu dikirim kembali. Penghapusan dilakukan dengan `splice()` di `roomModel.remove(id)`, dan karena tidak ada body, controller mengirim `res.status(204).send()`.



Bagian B. Skenario gagal

N01 Menambah data tanpa API key (POST /rooms)

Permintaan	POST /rooms (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ "nomorKamar": "105", "tipe": "standar", "hargaPerMalam": 400000, "kapasitas": 2, "tersedia": true }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah
Status hasil uji	401 Unauthorized; 'API key tidak valid atau tidak dikirim'
Lulus?	Ya

Gambar. POST tanpa API key ditolak

Penjelasan: Permintaan ditolak oleh middleware cekApiKey yang terpasang pada route POST di roomRoutes.js. Middleware berjalan sebelum fungsi controller, sehingga createRoom sama sekali tidak dijalankan dan data tidak bertambah. Ini membuktikan perlindungan route berjalan di layer middleware, bukan di controller.

N02 Menambah data dengan body tidak lengkap (POST /rooms)

Permintaan	POST /rooms
Header	x-api-key: revaldo Content-Type: application/json
Body	{ "nomorKamar": "105" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib
Status hasil uji	400 Bad Request; 'Field nomorKamar, tipe, hargaPerMalam, dan kapasitas wajib diisi'
Lulus?	Ya

Gambar. POST dengan body tidak lengkap

Penjelasan: Validasi ditulis di layer controller melalui fungsi validasi() pada roomController.js, yang dijalankan sebelum model dipanggil. Status 400 dipilih karena kesalahan berasal dari input client (field wajib kosong), sehingga client yang harus memperbaiki permintaannya.



N03 Mengirim JSON yang rusak (POST /rooms)

Permintaan	POST /rooms
Header	x-api-key: revaldo Content-Type: application/json
Body	{ "nomorKamar": "105", } (koma berlebih setelah nilai terakhir)
Hasil yang diharapkan	400 Bad Request; pesan 'Format JSON tidak valid' (bukan halaman HTML)
Status hasil uji	400 Bad Request; 'Format JSON tidak valid'
Lulus?	Ya

Gambar. POST dengan JSON rusak

Penjelasan: JSON rusak dideteksi oleh `express.json()` yang melempar error bertipe `SyntaxError` (`err.type = 'entity.parse.failed'`). Error ini tidak ditangani di route mana pun, melainkan jatuh ke `errorHandler` terpusat di `middlewares/errorHandler.js`, yang mengubahnya menjadi respons 400 JSON rapi. Tanpa error handler, client akan menerima halaman HTML stack trace default Express.

N04 Mengambil ID yang tidak ada (GET /rooms/99)

Permintaan	GET /rooms/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON
Status hasil uji	404 Not Found; 'Data dengan id 99 tidak ditemukan'
Lulus?	Ya

Gambar. GET ID tidak ada

Penjelasan: Controller `getRoomById` memanggil `roomModel.getById(99)` yang mengembalikan `undefined` karena id 99 tidak ada di array. Karena hasilnya kosong, controller sendiri yang menyusun dan mengirim respons 404 dengan format JSON melalui `res.status(404).json()`. Alurnya: route -> controller -> model (kosong) -> controller membuat pesan 404.



Bab 5 Rekapitulasi Hasil Uji

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	Ya
T05	Menghapus data	204	204 No Content	Ya
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Tautan repositori	https://github.com/revaldofiqqihseptiansyah2428240110/si5c_kuis1_RevaldoFiqqihSeptiansyah.git
Pesan commit terakhir	konfigurasi app dan env
Folder routes, controllers, models, middlewares ada	Ya
.env tidak terunggah	Ya



(ada di .gitignore)	
.env.example ada	Ya

Bab 7 Refleksi

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan: Route hanya memetakan alamat ke fungsi, contoh: `router.post("/", cekApiKey, roomController.createRoom)` di `routes/roomRoutes.js`. Controller menangani request, validasi, dan response, contoh: fungsi `validasi()` dan `res.status(400).json(...)` di `controllers/roomController.js`. Model menyimpan dan mengolah data tanpa req/res, contoh: `rooms.push(baru)` di dalam fungsi `create()` pada `models/roomModel.js`. Dengan pemisahan ini, perubahan logika data tidak menyentuh kode HTTP dan sebaliknya.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan: Error handler terpusat membuat semua jenis error ditangani di satu tempat sehingga format respons konsisten dan kode route tetap bersih. Buktinya pada pengujian N03: JSON rusak yang dilempar `express.json()` otomatis menjadi respons 400 'Format JSON tidak valid' tanpa menulis penanganan di setiap route; begitu pula route yang tidak dikenal otomatis 404 lewat middleware `notFound`. Jika ditulis manual di tiap route, setiap route harus mengulang kode penanganan yang sama.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan topik Tugas 1 saya	Ya



2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

